

Experiment 8: Travelling Salesman Problem (TSP)

Aim

To implement the Travelling Salesman Problem using Python.

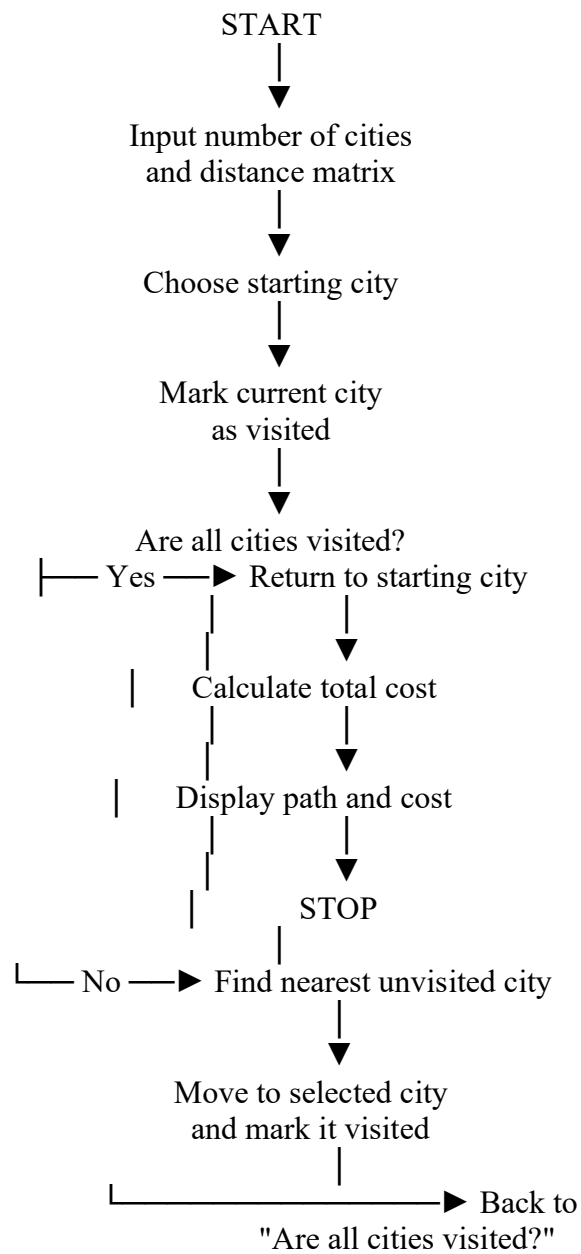
Objective

- To determine the minimum cost path.
- To visit every city exactly once and return to the starting city.

Algorithm

1. Store distances between cities.
2. Generate all possible routes.
3. Calculate the total cost for each route.
4. Select the route with minimum cost.
5. Display the optimal route.

Flowchart



Program

```
from itertools import permutations
```

```
graph = [
    [0,10,15,20],
    [10,0,35,25],
    [15,35,0,30],
    [20,25,30,0]
]
cities = [1,2,3]
min_cost = float('inf')
for path in permutations(cities):
    cost = graph[0][path[0]]
    for i in range(len(path)-1):
        cost += graph[path[i]][path[i+1]]
    cost += graph[path[-1]][0]
    if cost < min_cost:
        min_cost = cost
        best = path
print("Minimum Cost:", min_cost)
print("Route: 0 ->", *best, "-> 0")
```

Output

Programiz

Python Online Compiler

main.py

Share

Run

Output

Clear

```
1 from itertools import permutations
2 graph = [
3     [0,10,15,20],
4     [10,0,35,25],
5     [15,35,0,30],
6     [20,25,30,0]
7 ]
8 cities = ["I","J","K"]
9 min_cost = float('inf')
10 for path in permutations(cities):
11     cost = graph[path[0]]
12     for i in range(len(path)-1):
13         cost += graph[path[i]][path[i+1]]
14     cost += graph[path[-1]][path[0]]
15     if cost < min_cost:
16         min_cost = cost
17         best = path
18 print("Minimum Cost:", min_cost)
19 print("Route: 0 ->", " ".join(best), "-> 0")
```

Minimum Cost: 95
Route: 0 -> 1 2 3 -> 0
Minimum Cost: 80
Route: 0 -> 1 3 2 -> 0
Minimum Cost: 80
Route: 0 -> 1 3 2 -> 0
Minimum Cost: 80
Route: 0 -> 1 3 2 -> 0
Minimum Cost: 80
Route: 0 -> 1 3 2 -> 0
Minimum Cost: 80
Route: 0 -> 1 3 2 -> 0

=== Code Execution Successful ===

Result

The minimum travelling cost was obtained successfully.

Conclusion

The Travelling Salesman algorithm finds the shortest possible tour covering all cities.